

# Lab Practice Sheet 2

CS G526: Advanced Algorithms & Complexity

Instructor: Tulasimohan Molli

Date: 2026-09-13

## ! Evaluation Instructions for Students

- **Practice all 8 problems below.**
- In the next lab session, **one question will be chosen at random** for on-the-spot coding and evaluation.
- Your code will be evaluated by running it against automated hidden test cases.
- You may write your solutions in **Python**, **C++**, or **Java**.
- Standard data structure libraries are permitted unless a problem explicitly requires in-place or specific structural manipulation.

## Q1: The Palindrome Check

**Description:** Given a sequence of  $N$  integers representing a singly linked list / sequence, determine whether the sequence forms a palindrome (reads identically forwards and backwards).

### • Input Format:

- Line 1: An integer  $N$ , the number of elements.
- Line 2:  $N$  space-separated integers representing the sequence.

### • Output Format:

- A single line printing **True** if the sequence is a palindrome, or **False** otherwise.

### i Sample Input 1

```
5
1 2 3 2 1
```

### i Sample Output 1

```
True
```

### i Sample Input 2

```
4
1 2 3 4
```

### i Sample Output 2

```
False
```

## Q2: Rotate List to the Right by K Places

**Description:** Given a sequence of  $N$  integers and an integer  $K$ , rotate the sequence cyclically to the right by  $K$  positions. That is, each element moves forward by  $K$  positions, and elements that shift past the end wrap around to the beginning.

- **Input Format:**

- Line 1: An integer  $N$ , the number of elements.
- Line 2:  $N$  space-separated integers representing the sequence.
- Line 3: An integer  $K$ , the number of positions to rotate right.

- **Output Format:**

- A single line containing  $N$  space-separated integers representing the rotated sequence.

### Sample Input

```
5
10 20 30 40 50
2
```

### Sample Output

```
40 50 10 20 30
```

### Q3: Swap Nodes / Elements in Pairs

**Description:** Given a sequence of  $N$  integers, swap every two adjacent elements. If the number of elements is odd, the last element remains in its original position.

- **Input Format:**

- Line 1: An integer  $N$ , the number of elements.
- Line 2:  $N$  space-separated integers representing the sequence.

- **Output Format:**

- A single line containing  $N$  space-separated integers representing the pairwise swapped sequence.

#### **i** Sample Input

```
4
10 20 30 40
```

#### **i** Sample Output

```
20 10 40 30
```

**Q4: Filter Strictly Increasing Sequence from First Element**

**Description:** Given a sequence of  $N$  integers, scan from left to right starting at the first element. Eliminate any element that is less than or equal to its immediate predecessor in the retained sequence, preserving only elements that are strictly greater than the preceding kept element.

- **Input Format:**

- Line 1: An integer  $N$ , the number of elements.
- Line 2:  $N$  space-separated integers representing the sequence.

- **Output Format:**

- A single line printing the filtered sequence with elements separated by `->`.

**i** Sample Input

```
4
5 3 6 1
```

**i** Sample Output

```
5 -> 6
```

**Q5: Find the Two Added Elements**

**Description:** You are given an initial sequence  $A$  of  $N$  integers. Another sequence  $B$  of size  $N + 2$  contains all elements of  $A$  (in arbitrary reshuffled order) along with exactly two newly inserted random positive integers. Identify the two new elements and print them in non-decreasing order.

- **Input Format:**

- Line 1: An integer  $N$ , the size of sequence  $A$ .
- Line 2:  $N$  space-separated integers representing sequence  $A$ .
- Line 3:  $N + 2$  space-separated integers representing sequence  $B$ .

- **Output Format:**

- A single line containing the two added integers in ascending order, separated by a space.

**i** Sample Input

```
3
1 2 3
3 6 2 3 1
```

**i** Sample Output

```
3 6
```

## Q6: Same Parity Match Count

**Description:** Given an array of  $N$  integers, sort the array in non-decreasing order. In the sorted array, count: 1. **even\_count**: The number of times an **even** value appears at an **even** index (0, 2, 4, ...). 2. **odd\_count**: The number of times an **odd** value appears at an **odd** index (1, 3, 5, ...).

Indices are 0-based.

- **Input Format:**

- Line 1: An integer  $N$ , the number of elements.
- Line 2:  $N$  space-separated integers.

- **Output Format:**

- A single line printing **even\_count,odd\_count**.

### **i** Sample Input

```
3
2 5 4
```

### **i** Sample Output

```
1,0
```

**Q7: Maximum Elements in Range Difference K**

**Description:** Given an array of  $N$  integers and an integer  $K$ , find the maximum number of elements that can be chosen from the array such that the difference between the maximum and minimum values among all chosen elements is at most  $K$  ( $\max - \min \leq K$ ).

- **Input Format:**

- Line 1: An integer  $N$ , the size of the array.
- Line 2: An integer  $K$ , the maximum allowable difference.
- Line 3:  $N$  space-separated integers representing the array.

- **Output Format:**

- A single line containing an integer representing the maximum number of elements satisfying the condition.

**i** Sample Input

```
5
1
2 1 1 3 2
```

**i** Sample Output

```
4
```

**Q8: Search in a Row-wise and Column-wise Sorted Matrix**

**Description:** Given an  $M \times N$  matrix where every row is sorted in non-decreasing order from left to right, and every column is sorted in non-decreasing order from top to bottom, search for a target value  $K$ .

Your algorithm must locate  $K$  in  $O(M + N)$  time.

- **Input Format:**

- Line 1: Two space-separated integers  $M$  and  $N$ , the matrix dimensions.
- Next  $M$  lines: Each line contains  $N$  space-separated integers representing a row of the matrix.
- Next line: An integer  $K$ , the target value to search for.

- **Output Format:**

- If  $K$  is found, print its 0-based row and column indices as **r c**. If multiple occurrences exist, output any valid coordinate.
- If  $K$  is not present, print **-1**.

**i** Sample Input 1

```
4 4
10 20 30 40
15 25 35 45
27 29 37 48
32 33 39 50
29
```

**i** Sample Output 1

```
2 1
```

**i** Sample Input 2

```
2 2
1 2
3 4
99
```

**i** Sample Output 2

```
-1
```